

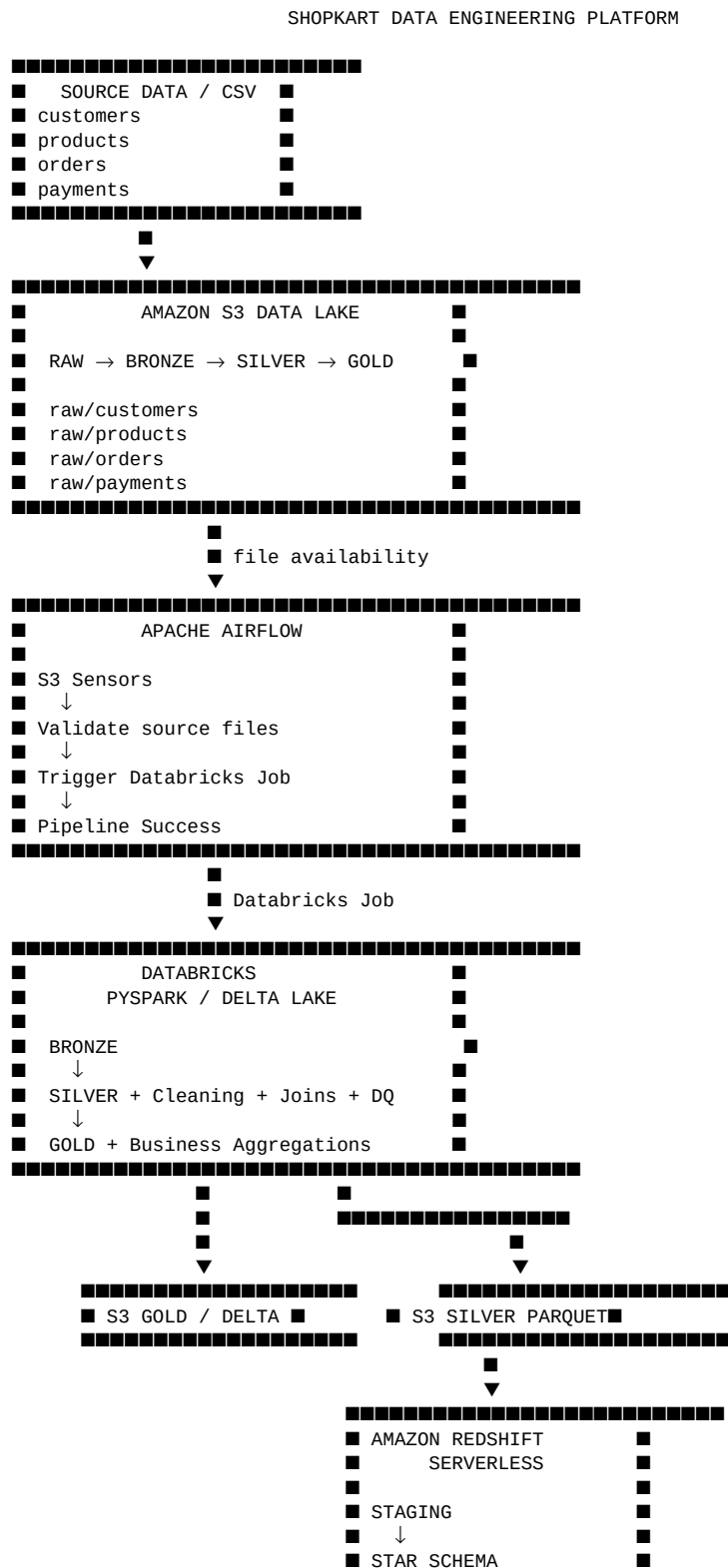
# ShopKart Data Engineering Platform

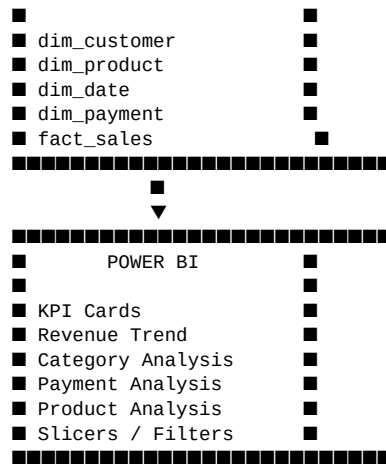
Complete Project Architecture & Data Flow

## End-to-End Cloud Data Engineering Project

AWS S3 → Apache Airflow → Databricks / PySpark → Delta Lake → Amazon Redshift → Power BI

## 1. Complete Project Architecture





## 2. Layer-by-Layer Architecture

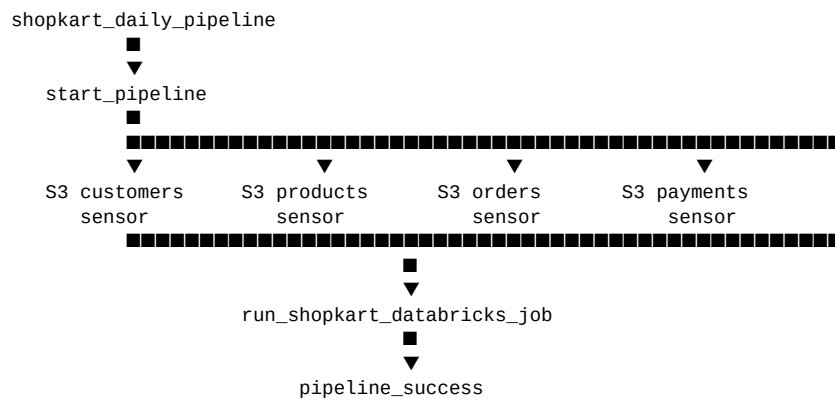
| Layer     | Technology                 | Purpose                                                   |
|-----------|----------------------------|-----------------------------------------------------------|
| Source    | CSV                        | Customers, products, orders and payments source data      |
| Raw       | Amazon S3                  | Original source files / landing zone                      |
| Bronze    | Databricks + Delta         | Raw data ingested into Delta format                       |
| Silver    | PySpark + Delta            | Cleaning, deduplication, joins, validation and enrichment |
| Gold      | PySpark + Delta            | Business-level aggregations and analytics outputs         |
| Warehouse | Amazon Redshift Serverless | Staging and dimensional/star-schema analytics             |
| BI        | Power BI                   | Interactive business dashboards                           |

## 3. Amazon S3 Data Lake

```

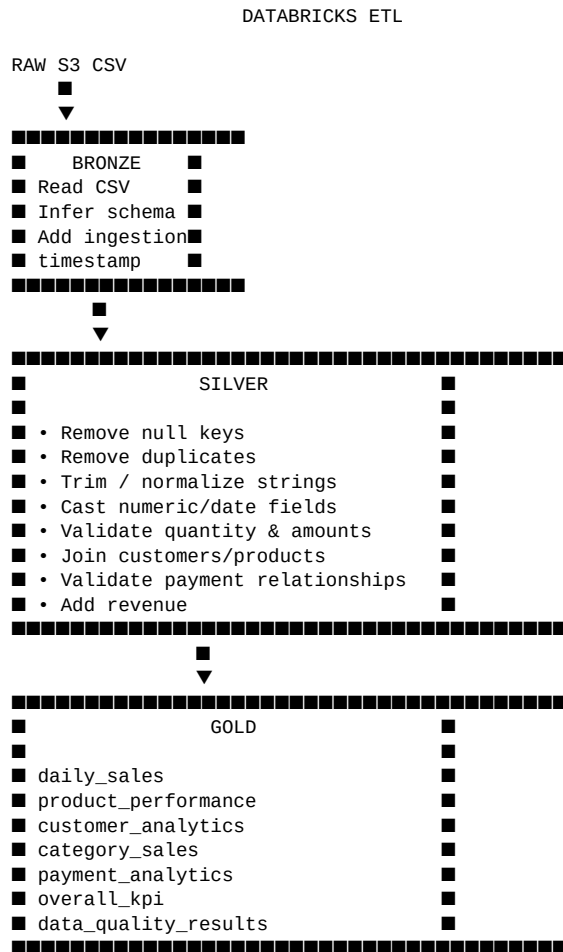
shopkart-data-2026/
├── raw/
│   ├── customers/customers.csv
│   ├── products/products.csv
│   ├── orders/orders.csv
│   └── payments/payments.csv
├── bronze/
│   ├── customers/
│   ├── products/
│   ├── orders/
│   └── payments/
├── silver/
│   ├── customers/
│   ├── products/
│   ├── orders/
│   └── payments/
├── gold/
│   ├── category_sales/
│   ├── customer_analytics/
│   ├── daily_sales/
│   ├── data_quality_results/
│   ├── overall_kpi/
│   ├── payment_analytics/
│   └── product_performance/
└── redshift/
    ├── silver_customers/
    ├── silver_products/
    ├── silver_orders/
    └── silver_payments/
  
```

## 4. Airflow Orchestration



The Airflow DAG runs daily, validates the presence of the four raw S3 objects using S3KeySensor, and triggers the Databricks job only after all required inputs are available. The configured DAG also uses retries and reschedule mode for the sensors.

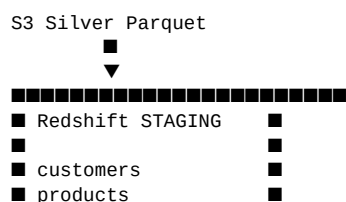
## 5. Databricks Processing Architecture

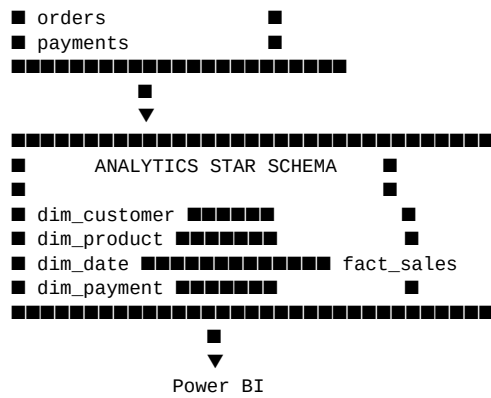


## 6. Data Quality Architecture

| Check                 | Purpose                                             |
|-----------------------|-----------------------------------------------------|
| Non-empty Silver      | Ensure transformed tables contain data              |
| Null keys             | Detect missing primary/business keys                |
| Duplicate IDs         | Detect duplicate customer/order/product/payment IDs |
| Referential integrity | Validate customer/product/order relationships       |
| Quantity > 0          | Prevent invalid order quantities                    |
| Amount >= 0           | Prevent invalid monetary values                     |
| Payment validation    | Remove/identify payments without valid orders       |
| Gold KPI sanity       | Validate final aggregate output                     |

## 7. Redshift Warehouse Architecture

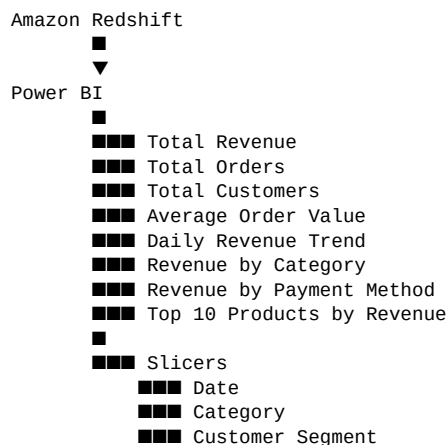




## 8. Star Schema

| Table        | Role               | Key / Important Fields                                                        |
|--------------|--------------------|-------------------------------------------------------------------------------|
| dim_customer | Customer dimension | customer_key, customer_id, segment, city, state                               |
| dim_product  | Product dimension  | product_key, product_id, category, subcategory, price                         |
| dim_date     | Date dimension     | date_key, full_date, year, quarter, month, week                               |
| dim_payment  | Payment dimension  | payment_key, payment_method, payment_status                                   |
| fact_sales   | Sales fact         | order_id, date_key, customer_key, product_key, payment_key, quantity, revenue |

## 9. Power BI Analytics Layer



## 10. Core Power BI Measures

```

Total Revenue =
SUM(fact_sales[revenue])

Total Orders =
DISTINCTCOUNT(fact_sales[order_id])

Total Customers =
DISTINCTCOUNT(fact_sales[customer_key])

Total Items =
SUM(fact_sales[quantity])

Average Order Value =
DIVIDE(
    [Total Revenue],
    [Total Orders],
    0
)
  
```

## 11. End-to-End Daily Execution

1. Source CSV files are available in S3 RAW
- ↓
2. Airflow starts daily DAG
- ↓
3. Four S3 sensors verify required files
- ↓
4. Airflow triggers Databricks job
- ↓
5. Bronze ingestion
- ↓
6. Silver cleaning + enrichment + relationships
- ↓
7. Data Quality validation
- ↓
8. Gold analytics generation
- ↓
9. Silver Parquet exported/loaded into Redshift
- ↓
10. Redshift staging populated
- ↓
11. Star schema populated
- ↓
12. Power BI reads Redshift analytics
- ↓
13. Business dashboard available

## 12. Technology Stack

| Area | Technology |
|------|------------|
|------|------------|

|                            |                            |
|----------------------------|----------------------------|
| Cloud                      | Amazon Web Services (AWS)  |
| Object Storage / Data Lake | Amazon S3                  |
| Orchestration              | Apache Airflow             |
| Processing                 | Databricks + PySpark       |
| Lakehouse Format           | Delta Lake                 |
| Warehouse                  | Amazon Redshift Serverless |
| Analytics                  | Power BI                   |
| Languages                  | Python, PySpark, SQL, DAX  |
| Version Control            | Git / GitHub               |

## 13. GitHub Repository Architecture

```
ShopKart-Data-Engineering-Platform/
├── aws/
│   ├── README.md
│   ├── s3_structure.md
│   ├── iam_setup.md
│   ├── redshift_setup.md
│   ├── sql/
│   └── screenshots/
├── airflow/
│   ├── dags/
│   │   └── shopkart_daily_pipeline.py
│   └── docker-compose.yaml
├── databricks/
│   ├── 01_bronze.py
│   ├── 02_silver.py
│   ├── 03_gold.py
│   └── 04_data_quality.py
├── redshift/
│   ├── 01_staging_tables.sql
│   ├── 02_dimension_tables.sql
│   ├── 03_fact_sales.sql
│   └── 04_validation.sql
├── powerbi/
│   ├── ShopKart_Sales_Dashboard.pbix
│   └── screenshots/
├── architecture/
│   └── architecture.png
├── dataset/
│   └── README.md
├── README.md
└── .gitignore
```

## 14. Security Rules

The GitHub repository should contain source code, SQL, documentation, architecture and selected screenshots—not credentials. AWS access keys, Databricks tokens, Redshift passwords, .env files and other secrets must remain outside the repository.

## 15. Project Outcome

The completed architecture demonstrates a complete cloud data engineering workflow: data landing in S3, automated orchestration with Airflow, distributed transformation with Databricks and PySpark, layered Delta Lake processing, data-quality validation, analytical warehousing in Redshift, and business reporting through Power BI.

## 16. Interview Summary

“I built an end-to-end ShopKart data engineering platform. Source CSV data lands in Amazon S3. Apache Airflow orchestrates the pipeline and validates the source files before triggering Databricks. Databricks uses PySpark and Delta Lake to implement Bronze, Silver and Gold layers, including data quality checks and business aggregations. Silver data is then made available to Amazon Redshift Serverless, where I built a staging layer and a star schema with customer, product, date and payment dimensions and a sales fact table. Finally, Power BI connects to the warehouse for interactive business analytics.”

This architecture document is based on the ShopKart project structure and implementation details established in the project workflow. It is intended to accompany the AWS, Airflow, Databricks, Redshift and Power BI phase documentation in the GitHub repository.